



Project Overview

This major task requires students to develop a complete software application using the programming concepts learned throughout the course. Each team will choose one of the three available projects and deliver it across two milestones.

Team Requirements

- Students work in groups of **exactly four (4) members**
- Each group chooses **one** of the three projects

Milestone Schedule

Milestone	Deadline	Weight
Milestone 1	Week 12	10 marks
Milestone 2	Week 14	15 marks
Total		25 marks

Evaluation Criteria

Milestone 1		Milestone 2	
Feature	Weight	Feature	Weight
Design Documentation	2.5	Complete Functionality	6
Partial Implementation	4	Input Validation	2
Input Validation	2	Data Persistence	2.5
Code Structure	1	User Interface	2
Report Quality	0.5	Documentation	1.5
		Code Efficiency	0.5
		Final Report	0.5

General Instructions

1. **Plagiarism = zero marks** for entire team
2. Use only course material (except file handling)
3. Submit: Technical report with code in appendix
4. All team members must understand and be able to explain the entire codebase
5. Team members may receive different grades based on individual understanding and contributions

Project 1: Bank Customers Queuing System

Objective

Simulate a bank queue: customers take tickets, wait for service, and get processed by tellers. Track wait times and generate reports.

Data Structures

- Customer:** ID (auto-generated), Arrival Time (HH:MM), Transaction Type, Service Start/End Time, Served Status (Yes/No)
- Teller:** ID (T001, T002...), Assigned Transactions, Status (Available/Busy/On Break)
- Configuration:** Working hours (default 9AM-3PM), Number of tellers per day, Yearly customer target (>30,000)

Required Features

#	Feature	Description
1	System Configuration	Set tellers per day, working hours, transaction types, yearly target
2	Run Simulation	Daily/Weekly/Yearly simulation with random customers, FIFO processing
3	View Reports	Customer report (arrivals, service times) and Teller report (utilization)
4	View Statistics	Total customers, avg wait time, peak hours, busiest day

Validation Rules

- Tellers: 1 to MAX_TELLERS (default 10)
- Time format: HH:MM, opening < closing
- Transaction names: unique, non-empty; duration > 0
- Yearly target: integer > 30,000

Files (bonus)

customers_report.txt, tellers_report.txt

Milestones

Milestone 1 (Week 12)	Milestone 2 (Week 14)
Design docs (architecture, use cases, data flow)	Complete simulation (Daily/Weekly/Yearly)
Main menu + System Configuration	View Reports + View Statistics
All Operation 1 validations	Export to files (bonus)
Save/load configuration	Full data persistence
Technical report + pseudocode	Final report + user manual

Project 2: Student Information System (SIS)

Objective

Manage student records, courses, and grades. Support CRUD (create, read, update, and delete) operations, GPA calculation, and transcript generation.

Data Structures

- Student:** ID (YYPXXXX), Name (two words), National ID (14 digits, unique), Gender (M/F), DOB (DD/MM/YYYY, age≥17), Phone (01xxxxxxxxx), Program (CSE/CCE/MCT), Level (1-4), GPA (0.0-4.0)
- Course:** Code (3 letters + 3 digits), Title, Credit Hours (2-4)
- Grade:** Student ID, Course Code, Midterm (0-40), Final (0-60)

Required Features

#	Feature	Description
1	Student Management	Add, Search (by ID/Name/NationalID), Update, Delete, List (sort by ID/Name/GPA)
2	Course Management	Add, View, Update, Delete courses
3	Grades Management	Enter grades, View grades, Calculate GPA, Generate Transcript

Validation Rules

- Student ID: YYPXXXX format, unique
- National ID: 14 digits, no leading zero, unique
- Name: exactly two words
- Phone: 11 digits starting with "01"
- Course Code: 3 letters + 3 digits, unique

Files (bonus)

students.txt, courses.txt, grades.txt

Milestones

Milestone 1 (Week 12)	Milestone 2 (Week 14)
Design docs (architecture, use cases, data flow)	Update/Delete Student
Main menu + Student Management (Add, Search, List)	Course Management (full CRUD)
All student field validations	Grades Management + GPA + Transcript
Save/load students.txt (bonus)	All 3 files with auto-load/save (bonus)
Technical report + pseudocode	Final report + user manual

Project 3: Study Schedule Generator

Objective

Generate conflict-free timetables for all programs and levels. Manage rooms, subjects, instructors, and scheduling constraints.

Data Structures

- **Room:** ID (1 letter + 2 digits), Name, Type (Lecture/Lab/Tutorial), Capacity
- **Subject:** Code (3 letters + 3 digits), Name, Lecture/Tutorial/Lab hours (total > 0), Instructor
- **Section:** Program (CSE/CCE/MCT), Level (1-4), Section Count

Required Features

#	Feature	Description
1	Room Management	Add, View, Update, Delete rooms
2	Subject Management	Add, View, Update, Delete subjects
3	Section Management	Add, View, Update, Delete sections
4	Instructor Preferences	Set/View/Clear unavailable time slots
5	Generate Schedule	Validate data, generate conflict-free schedule
6	View Schedule	View by Program/Instructor/Room, Export to file

Scheduling Rules

1. No Instructor Conflict (can't teach 2 sessions simultaneously)
2. No Room Conflict (no double-booking)
3. No Student Conflict (no overlapping sessions for same group)
4. Room Type Match (labs in lab rooms only)
5. Instructor Availability (respect unavailable slots)
6. Gap Minimization (≤ 2 free slots per day per group)

Files (bonus)

rooms.txt, subjects.txt, sections.txt, preferences.txt, schedule.txt

Milestones

Milestone 1 (Week 12)	Milestone 2 (Week 14)
Design docs + scheduling algorithm pseudocode	Section Management
Main menu + Room Management	Instructor Preferences
Subject Management	Generate Schedule (all 6 rules)
All Room/Subject validations	View Schedule
Save/load rooms & subjects (bonus)	All 5 files with auto-load/save (bonus)
Technical report + pseudocode	Final report + user manual